Validation with pure Java

Build a solid foundation for the data-validation framework with the core Java API

Summary
The importance of employing a good data-validation framework cannot be overestimated. Eventually, someone (maybe even yourself!) is going to reuse the classes you've created. Unless your objects come with a built-in, robust data-validation mechanism that requires little maintenance, you'll need to advise that someone about the valid data ranges accepted by those objects. The core Java API has everything you need to solve this problem in the most elegant way. In this article, you will build a foundation for the data-validation framework whose grown-up version has been implemented successfully in large-scale commercial projects. *(1,500 words)*

By Victor Okunev

The idea of *constrained* properties in Java is not new. Since JDK 1.1, a legion of JavaBeans developers has used a powerful framework found in the `java.beans` package to enforce data-validation rules. Unfortunately, the rest of us -- who are not in the business of crafting "reusable software components that can be manipulated visually in a builder tool" -- quite often attempt to reinvent the wheel by abandoning the core Java approach in favor of various proprietary solutions. The externalization of data-validation rules is the area where you can see the most creativity. An interesting XML-based approach was recently proposed in the "Validation with Java and XML Schema" series of *JavaWorld*. While admiring XML technology, I believe that Java has everything needed to solve the issue in the most elegant way. To show you, I invite you to rediscover some of the real gems found in the `java.beans` package. They will help you build a few useful classes and learn a couple of interesting tricks.

The logic behind constrained properties in Java is quite simple. Before accepting a new data value, the object (owner of the constrained property) makes sure it is accepted by all interested parties that may veto it. Such a "request for approval" is delivered to every registered `java.beans.VetoableChangeListener` in the form of a `java.beans.PropertyChangeEvent` object. If one or more vetoes have been issued, the proposed data value is rejected. A veto is presented by a `java.beans.PropertyVetoException`. Generally speaking, the complexity of the rules enforced by those listeners has no limit and depends only on the project's requirements and the developer's creativity. The objective of this exercise is to learn how to deal with the most generic data-validation criteria that you can apply to most objects.

First, let's create a class called `BusinessObject` with a simple numeric property. Next, I'll show

how you can modify the property into a constrained one to see how it differs from the simple one.

```
public class BusinessObject {

  private int numericValue;

  public void setNumericValue(int newNumericValue) {
    numericValue = newNumericValue;
  }

  public int getNumericValue() {
    return numericValue;
  }
}
```

Please note that the only property of this class will silently accept any value of the proper type. To make it constrained, a vetoable event needs to be produced and distributed to the consumers. A utility class called `java.beans.VetoableChangeSupport` was designed specifically to fulfill this set of responsibilities. A new class called `ConstrainedObject` will manage all the arrangements with this utility class, thereby being a good loving parent for the `BusinessObject`. Speaking of consumers, we need to build another custom class called `Validator` that hosts generic data-validation algorithms. It is going to be the only consumer of the vetoable events in our framework. This simplified approach deviates from the rules of the JavaBeans game (check JavaBeans API Specification for details), which requires presenting every constrained property as a bound one and providing support for handling multiple listeners. This deviation is perfectly acceptable, since you are not building a JavaBean here, but it's still worth mentioning.

```
import java.beans.*;

/**
* The responsibility of the ConstrainedObject is to delegate generic data
entry
* validation responsibilities to the {@link Validator} and provide
* subclasses with transparent interface to it.
*/
public class ConstrainedObject {

  private VetoableChangeSupport vetoableSupport = new VetoableChangeSupport
(this);

  /**
   * Creates a new object with generic property validator
   */
  public ConstrainedObject() {
    vetoableSupport.addVetoableChangeListener(new Validator());
  }

  /**
   * This method will be used by subclasses to validate a new value in
   * the constrained properties of the int type. It can be easily overloaded
   * to deal with other primitive types as well.
   * @param propertyName The programmatic name of the property that is about
```

```
to change.
   * @param oldValue The old value of the property.
   * @param newValue The new value of the property.
   */
  protected void validate(String propertyName, int oldValue, int newValue)
throws PropertyVetoException {
    vetoableSupport.fireVetoableChange(propertyName, new Integer(oldValue),
new Integer(newValue));
  }
}
```

For the sake of keeping the code compact, the `ConstrainedObject` provides a method that validates `int` properties only. You can easily overload this method for the other types as well. If you do so, just make sure to wrap all the primitive types in their respective wrappers since `PropertyChangeEvent` delivers old and new values as references types. Now you are ready to issue a second revision of the `BusinessObject`:

```
import java.beans.*;

/**
* This example class actually uses data-validation services defined in this
framework.
*/
public class BusinessObject extends ConstrainedObject {

  private int numericValue;

  public void setNumericValue(int newNumericValue) throws
PropertyVetoException {
    // validate proposed value
    validate("numericValue", numericValue, newNumericValue);
    // the new value is approved, since no exceptions were thrown
    numericValue = newNumericValue;
  }

  public int getNumericValue() {
    return numericValue;
  }
}
```

As you see, the setter method now looks a bit more complicated than before. It is a small price to pay for having a constrained property. Now you must build a `Validator`, which is the most interesting part of this exercise. This is where you learn how to externalize data-validation rules using nothing but pure Java.

```
import java.beans.*;

public class Validator implements VetoableChangeListener {

  public void vetoableChange(PropertyChangeEvent evt) throws
PropertyVetoException {
    // do the validation here
```

```
  }
}
```

That code skeleton is just a starting point, but it demonstrates what is available at the beginning of the validation process. Surprisingly, it has a lot of information. From `PropertyChangeEvent` you can learn about the constrained property's object source, the name of the property itself, and its existing and proposed values. With that information handy, you can use the power of *introspection* to inquire about the property's additional information. What information should you look for? Validation rules, of course! A class `java.beans.Introspector` was designed solely for the purpose of collecting extra information about a target class. The information is provided in the form of a `java.beans.BeanInfo` object, which in this example, you may request with a simple call like this:

```
BeanInfo info = Introspector.getBeanInfo(evt.getSource().getClass());
```

The `Introspector` may collect `BeanInfo` in two ways. First it tries to get explicit information about the target class by looking for a class with the same name except for its `BeanInfo` suffix. In this case, it will look for a class named `BusinessObjectBeanInfo`. If for any reason such a class is not available, then the `Introspector` tries its best to dynamically build a `BeanInfo` object by using low-level reflection.

Although a fascinating subject, the implicit introspection is obviously not what you are going to rely upon in your quest for validation rules. One great thing about the `BeanInfo` class is its relation to the target class, which can be defined as a "rock-solid loose coupling." The absence of references from the target class makes the coupling very loose. Should the validation rules change, you just need to update and recompile the `BeanInfo` class without possibly affecting the target class. Clearly defined naming rules that are documented in the core Java API make compiling rock solid. This is where the rules-externalization approach with `BeanInfo` beats its proprietary-implemented counterparts hands down.

Before you start building the explicit `BeanInfo` class, I have to mention that you can load it with a variety of descriptors concerning the class itself, or its methods and events, not just the properties. Thanks to the creators of Java, you don't have to provide all that information. The `Introspector` picks up what is available explicitly and then performs its implicit analysis magic. Therefore, you can concentrate on the property descriptors only.

To describe a property's data-validation rules, you use the `java.beans.PropertyDescriptor` class. Looking at the documentation, you'll discover that this class has the facilities to provide every imaginable piece of information about the property, including data-validation rules! You use the `setValue` method to define the rules as a set of named attributes. Here is how you can constrain the bounds for the numeric property `numericValue`:

```
PropertyDescriptor _numericValue = new PropertyDescriptor("numericValue",
targetClass, "getNumericValue", "setNumericValue");
_numericValue.setValue("maxVal", new Integer(100));
_numericValue.setValue("minVal", new Integer(-100));
```

The only weak point here is that you cannot rely on the power of the Java compiler to check the spelling of the attribute's names. Using a predefined set of constants can easily fix that. The class `Validator` is a good candidate for hosting such constants:

```
public static final String MAX_VALUE = "maxValue";
public static final String MIN_VALUE = "minValue";
```

So now, you can rewrite the same rules more safely. With all this in mind, let's build a final revision of the `BeanInfo` class:

```
import java.beans.*;

public class BusinessObjectBeanInfo extends SimpleBeanInfo {

  Class targetClass = BusinessObject.class;

  public PropertyDescriptor[] getPropertyDescriptors() {
    try  {
      PropertyDescriptor numericValue = new PropertyDescriptor
("numericValue", targetClass, "getNumericValue", "setNumericValue");
      numericValue.setValue(Validator.MAX_VALUE, new Integer(100));
      numericValue.setValue(Validator.MIN_VALUE, new Integer(-100));

      PropertyDescriptor[] pds = new PropertyDescriptor[] {numericValue};
      return pds;

    } catch(IntrospectionException ex) {
      ex.printStackTrace();
      return null;
    }
  }
}
```

Finally, you complete the `Validator` so it can retrieve and analyze the information. It will use a handy method for retrieving a `PropertyDescriptor` from the `BeanInfo` by the property's programmatic name. In case the property was not found for any reason, the method will notify the caller with a self-explanatory exception.

```
import java.beans.*;

/**
* This class implements generic data-validation mechanism that is based on
the external
* rules defined in the BeanInfo class.
*/
public class Validator implements VetoableChangeListener {

  public static final String MAX_VALUE = "maxValue";
  public static final String MIN_VALUE = "minValue";

  /**
   * The only method required by {@link VetoableChangeListener} interface.
   */
  public void vetoableChange(PropertyChangeEvent evt) throws
PropertyVetoException {
```

```
    try {

      // here we hope to retrieve explicitly defined additional information
      // about the object-source of the constrained property
      BeanInfo info = Introspector.getBeanInfo(evt.getSource().getClass());

      // find a property descriptor by given property name
      PropertyDescriptor descriptor = getDescriptor(evt.getPropertyName(),
info);

      Integer max = (Integer) descriptor.getValue(MAX_VALUE);

      // check if new value is greater than allowed
      if( max != null && max.compareTo(evt.getNewValue()) < 0 ) {
        // complain!
        throw new PropertyVetoException("Value " + evt.getNewValue() + " is
greater than maximum allowed " + max, evt);
      }

      Integer min = (Integer) descriptor.getValue(MIN_VALUE);

      // check if new value is less than allowed
      if( min != null && min.compareTo(evt.getNewValue()) > 0 ) {
        // complain!
        throw new PropertyVetoException("Value " + evt.getNewValue() + " is
less than minimum allowed " + min, evt);
      }

    } catch (IntrospectionException ex) {
      ex.printStackTrace();
    }
  }

  /**
   * This utility method tries to fetch a PropertyDescriptor from BeanInfo
object by given property
   * name.
   * @param name the programmatic name of the property
   * @param info the bean info object that will be searched for the property
descriptor.
   * @throws IllegalArgumentException if a property with given name does not
exist in the given
   * BeanInfo object.
   */
  private PropertyDescriptor getDescriptor(String name, BeanInfo info)
throws IllegalArgumentException {

    PropertyDescriptor[] pds = info.getPropertyDescriptors();

    for( int i=0; i<pds.length; i++) {
      if( pds[i].getName().equals(name) ) {
        return pds[i];
      }
```

```
    }

    throw new IllegalArgumentException("Property " + name + " not found.");
  }
}
```

Well, believe it or not, that's it! You have all the pieces in place now and are ready to test your data validation. This simple driver does just that:

```
/**
* A simple application that tries to cause data-validation exception
* condition.
*/
public class Driver {

  public static void main(String[] args) {

    try {
      BusinessObject source = new BusinessObject();
      source.setNumericValue(10);
      System.out.println("Value accepted: " + source.getNumericValue());
      source.setNumericValue(-10);
      System.out.println("Value accepted: " + source.getNumericValue());
      source.setNumericValue(101);
      // we should never reach this line
      System.out.println("Value accepted: " + source.getNumericValue());
    } catch (Exception ex) {
      ex.printStackTrace();
    }
  }
}
```

When you run the application, you will see that the line `source.setNumericValue(101)` causes a `PropertyVetoException` with the following message: "Value 101 is greater than maximum allowed 100." Frankly, it is not often that exceptions make me happy. I guess this case is an exception.

Conclusion

In this short article, we have achieved quite a lot. I have shown you that the core Java API has everything you need to build a simple and robust data-validation mechanism. In addition, I've shown you how to build this mechanism into a framework whose grown-up version I successfully implemented in actual large-scale commercial projects. Of course I do not "hand bomb" either `BusinessObject` or `BeanInfo` classes, since the number of properties in the financial business object I deal with sometimes reaches a couple hundred. With my team, I have built an application that automatically generates Java code for those classes based on the information from the data dictionary. With this approach it takes no time at all to adjust the rules if needed. Looking at the code you may notice that the only package you had to import was `java.beans`. I hope that you will have a greater appreciation for it after reading this article. Please feel free to modify and improve the classes built here. After all, it is just a proof of concept.

About the author

Sun Certified Java Programmer Victor Okunev is a design architect of the R&D team for Plexus

Systems Design of Vancouver. With an M.S. in computer science from Moscow State Institute of Radio Engineering, Electronics, and Automation, Okunev has more than 10 years' experience in software development, primarily on enterprise-scale applications. He also teaches Java classes for Learning Tree International, snowboards, and enjoys sea and whitewater kayaking.

Resources

- Download the source code to this article: http://www.javaworld.com/jw-12-2000/valframe/valframe.zip
- Learn the JavaBeans concepts from the best source: JavaBeans API Specification: http://java.sun.com/products/javabeans/docs/spec.html
- Use this online tutorial to get up to speed with the content of `java.beans` package: http://java.sun.com/docs/books/tutorial/javabeans/index.html
- The series "Validation with Java and XML Schema" by Brett McLaughlin describes an XML approach for validating data with Java:
  - Part 1. Learning the Value of Data Validation and Why Pure Java Isn't the Complete Solution for Handling It
  - Part 2. Using XML Schema for Constraining Java Data
  - Part 3. Parsing XML Schema to Validate Data
  - Part 4. Building Java Representations of Schema Constraints and Applying Them to Java Data
- For a complete listing of Mark Johnson's **JavaBeans** columns in *JavaWorld*: http://www.javaworld.com/javaworld/topicalindex/jw-ti-beans.html
- For a complete listing of all **JavaBeans**-related articles on *JavaWorld*: http://www.javaworld.com/javaworld/topicalindex/jw-ti-jbeans.html
- Sign up for the *JavaWorld This Week* free weekly email newsletter and keep up with what's new at *JavaWorld*: http://www.idg.net/jw-subscribe